

Xcode Bracket Commenting Extension Project

1. Problem Statement

When developing in Xcode, developers frequently need to temporarily disable code blocks (such as `withAnimation`, `HStack`, etc.). The current workflow is cumbersome:

- Developer must manually comment out the opening line
- Developer must then locate and comment out the matching closing bracket
- Finding the correct closing bracket is tedious and error-prone, especially with nested blocks
- No built-in Xcode command exists to handle this common operation

Example of affected code:

```
swift withAnimation { // Developer wants to disable this block if isIcon { // restore window size let oldMask = floatWin.myWin.styleMas
```

2. Functional Description of Solution

Create an Xcode Source Editor Extension that provides the following functionality:

Core Features

- Activates when cursor is on a line containing an opening or closing bracket
- Automatically identifies the matching bracket
- Comments out (or uncomments) both the opening and closing lines
- Can be assigned to a keyboard shortcut

User Experience

- Single command execution (one keyboard shortcut)
- Works with any type of code block (functions, if statements, SwiftUI view modifiers, etc.)
- Maintains code indentation
- Provides clear feedback if operation cannot be completed

Optional Enhancements

- Visual preview of what will be commented
- Configuration options for comment style

3. Implementation Task Outline

A. Development Setup

1. Create new Xcode Source Editor Extension target
2. Configure extension entitlements
3. Set up Info.plist entries for extension
4. Configure command registration in Xcode

B. Core Implementation

1. Create primary command class
 - Implement `XCSourceEditorCommand` protocol
 - Set up command handling

2. Implement bracket matching algorithm

```
swift func findMatchingBracket(buffer: XCSourceTextBuffer, line: Int, column: Int) -> (line: Int, column: Int)?
```

- Parse from cursor position forward

- Track bracket depth for nested structures
- Handle different bracket types ({}, [], ())
- Return position of matching bracket

3. Create commenting logic `swift func toggleComment(buffer: XCSourceTextBuffer, startLine: Int, endLine: Int)`

- Add/remove comment markers
- Handle different indentation levels
- Preserve original formatting

4. Implement error handling

- Invalid cursor position
- Malformed code blocks
- Missing closing brackets

C. Testing & Validation

1. Create comprehensive test suite

- Different block types (if/else, functions, view modifiers)
- Nested structures
- Various indentation patterns
- Edge cases

2. Create sample code files for testing

- SwiftUI views
- Complex nested structures
- Different comment styles

D. User Interface Integration

1. Add command to Xcode's Editor menu
2. Configure keyboard shortcut capability
3. Implement user feedback
 - Success/failure indicators
 - Error messages
 - Status updates

E. Documentation

1. Write installation instructions
2. Create usage documentation
3. Document known limitations
4. Provide troubleshooting guide

Additional Resources

- [Apple's Source Editor Extension documentation](#)
- [Xcode Source Editor Extension sample code](#)
- Example extensions for reference (to be added)